

Encapsulation

In traditional OOP languages like C++, you hide data using access modifiers (private, protected, public). Go takes a much simpler approach: **Visibility is determined entirely by capitalization at the package level.**

1. The "Capital Letter" Rule (Exported vs. Unexported)

In Go, encapsulation happens at the package boundary, not the struct boundary.

- **Exported (Public):** If a **variable, function, struct, or struct field** starts with a **Capital Letter** (e.g., PrintX, Point, Scale), it is completely visible and accessible to any other package that imports it.
- **Unexported (Private):** If it starts with a **lowercase letter** (e.g., x, y, initMe), it is strictly hidden. It can only be seen and modified by code living inside the exact same package.

2. Controlled Access via Public Methods

If you hide all your data by making it lowercase, how does the rest of your application use it? You provide **Controlled Access** by writing Exported (public) methods that interact with the Unexported (private) data on behalf of the user.

- **The Setup:** You define a Point struct in a data package with hidden x and y fields.

```
package data
type Point struct {
    x float64 // hidden
    y float64 // hidden
}
```

- **The API:** You provide capitalized methods so outside code can safely interact with the point.

```
// Public method to safely set the hidden coordinates
func (p *Point) InitMe(newX, newY float64) {
    p.x = newX
    p.y = newY
}

// Public method to safely modify the hidden coordinates
func (p *Point) Scale(factor float64) {
    p.x = p.x * factor
    p.y = p.y * factor
}
```

3. Why do we do this?

By forcing outside packages to use your **Scale()** method instead of letting them manually write **p.x = 10**, you guarantee that your data remains perfectly consistent. You ensure that if **x** is scaled, **y** is always scaled with it, completely removing the chance of human error from the developers using your package.